

Informe Técnico: Pipeline de Streaming en Tiempo Real

Procesamiento Escalable con Apache Beam y Apache Kafka

Pilar Ruiz Diaz • Mathias Chaparro • Tanya Godoy

21 de septiembre de 2026

1. Resumen Ejecutivo

El presente documento describe el diseño, arquitectura e implementación de una solución de procesamiento de datos en *streaming* en tiempo real. La plataforma ingiere eventos sintéticos en formato JSON emitidos dinámicamente mediante Apache Kafka, procesa los datos aplicando agregaciones por ventanas temporales fijas (*Fixed Windows*) y eliminación de duplicados (*deduplication*) mediante Apache Beam en Python, y publica los resultados de forma idempotente en un tópico de destino.

2. Diagrama y Arquitectura del Sistema

El flujo de datos (*end-to-end*) sigue la arquitectura descrita a continuación:

Productor Sintético $\rightarrow$ *Kafka*(eventos-entrada) $\rightarrow$ *ApacheBeam*(Ventanas + Deduplicación) $\rightarrow$ *Kafka*(eventos-salida)

2.1. Flujo Detallado del Pipeline

- Productor Sintético:** Generación e inyección continua de mensajes JSON en tiempo real.
- Ingesta (eventos-entrada):** Tópico de lectura en Apache Kafka desplegado sobre contenedores Docker.
- Procesamiento en Apache Beam (DirectRunner):**
 - Parseo del *payload* JSON y extracción de la clave de evento (`event_id`).
 - Asignación de Ventanas Temporales Fijas de 10 segundos (*FixedWindows*).
 - Agrupación de registros por clave de evento (`GroupByKey`).
 - Filtrado y deduplicación a través de `DeduplicateDoFn`.
- Destino (eventos-salida):** Escritura idempotente del evento consolidado en Apache Kafka asignando `event_id` como *partition key*.

2.2. Componentes de Infraestructura

- **Gestor de Coordinación:** Apache ZooKeeper en Docker para la gestión de estados.
- **Plataforma de Eventos:** Apache Kafka ejecutándose en el puerto local 9092.
- **Motor de Procesamiento:** Apache Beam ejecutado sobre DirectRunner bajo un entorno virtual de Python 3.9.
- **Cliente de Integración:** Librería nativa `confluent-kafka` para evitar problemas de compatibilidad con Java Expansion Services en entornos Windows.

3. Especificaciones Técnicas de la Implementación

3.1. Fuente de Datos

La ingesta se realiza a partir de un **productor de eventos sintéticos** en formato JSON enviando estructuras de datos en tiempo real. Los eventos contienen la siguiente estructura base:

```
1 {  
2   "event_id": "EV100",  
3   "timestamp": "2026-09-21T18:45:00Z",  
4   "user_id": "U500",  
5   "action": "checkout"  
6 }
```

Listing 1: Ejemplo de payload sintético enviado al tópico de entrada

3.2. Ventanas Temporales y Deduplicación

Para el procesamiento en tiempo real se implementó una estrategia en tres fases dentro del *pipeline*:

1. **Ventana Fija (*Fixed Window*):** Los mensajes entrantes se agrupan en ventanas de tiempo discretas de 10 segundos de duración.
2. **Agrupación por Clave (GroupByKey):** Los eventos dentro de cada ventana se agrupan asociando el `event_id` como clave principal:

$$\text{Entrada} \rightarrow (\text{event_id}, [\text{evento}_1, \text{evento}_2, \dots])$$

3. **Filtro de Deduplicación (DeduplicateDoFn):** Se selecciona únicamente la primera instancia del grupo dentro de la ventana, descartando réplicas generadas por reintentos de red.

3.3. Escritura Idempotente

El sumidero (*sink*) utiliza la clase `KafkaWriteDoFn`, la cual reenvía el *payload* JSON procesado al tópico `eventos-salida`. Para garantizar **idempotencia**, la clave de partición (*partition key*) asignada en la publicación de Kafka coincide con el `event_id` original, asegurando que escrituras repetidas sobre la misma ventana no generen incoherencias en el destino.

4. Estrategia de Pruebas y Validación

4.1. Pruebas Unitarias Automáticas (pytest)

Se diseñó un conjunto de pruebas automáticas en el archivo `test_pipeline.py` utilizando la utilería de pruebas de Apache Beam (`TestPipeline`). Se incluyen las siguientes comprobaciones:

- **Prueba 1** (`test_deduplication_logic`): Valida que ante un conjunto de entrada con registros que contienen el mismo `event_id`, la transformación entregue una colección deduplicada de una única ocurrencia.
- **Prueba 2** (`test_unique_events_pass_through`): Verifica que los eventos únicos sin duplicados dentro de la ventana atraviesen la transformación sin ser alterados.
- **Comando de ejecución:** `pytest test_pipeline.py`
- **Resultado Obtenido:** Ejecución exitosa de la suite de pruebas (2 passed).

```
venv\lib\site-packages\google\oauth2\_init__.py:40
C:\Users\tanya\OneDrive\Escritorio\proyecto\venv\lib\site-packages\google\oauth2\_init__.py:40: FutureWarning: You are using a Python version 3.9 past its end of life. Google will update google-auth with critical bug fixes on a best-effort basis, but not with any other fixes or features. Please upgrade your Python version, and then update google-auth.
  warnings.warn(eol_message.format("3.9"), FutureWarning)

venv\lib\site-packages\google\api_core\_python_version_support.py:246
C:\Users\tanya\OneDrive\Escritorio\proyecto\venv\lib\site-packages\google\api_core\_python_version_support.py:246: FutureWarning: You are using a non-supported Python version (3.9.13). Google will not post any further updates to google.cloud.bigquery_storage_v1 supporting this Python version. Please upgrade to the latest Python version, or at least Python 3.10, and then update google.cloud.bigquery_storage_v1.
  warnings.warn(message, FutureWarning)

venv\lib\site-packages\google\api_core\_python_version_support.py:246
C:\Users\tanya\OneDrive\Escritorio\proyecto\venv\lib\site-packages\google\api_core\_python_version_support.py:246: FutureWarning: You are using a non-supported Python version (3.9.13). Google will not post any further updates to google.pubsub_v1 supporting this Python version. Please upgrade to the latest Python version, or at least Python 3.10, and then update google.pubsub_v1.
  warnings.warn(message, FutureWarning)

venv\lib\site-packages\apache_beam\testing\test_pipeline.py:36
C:\Users\tanya\OneDrive\Escritorio\proyecto\venv\lib\site-packages\apache_beam\testing\test_pipeline.py:36: PytestCollectionWarning: cannot collect test class 'TestPipeline' because it has a __init__ constructor (from: test_pipeline.py)
  class TestPipeline(Pipeline):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 2 passed, 15 warnings in 3.60s =====
```

Figura 1: Evidencia de ejecución exitosa de la suite de pruebas unitarias con pytest (2 passed).

5. Estructura del Repositorio

```
1 proyecto/
2   -- docker-compose.yml          # Definición de contenedores Kafka y ZooKeeper
3   -- requirements.txt            # Dependencias (apache-beam, confluent-kafka, pytest)
4   -- pipeline.py                 # Código fuente principal del pipeline de streaming
5   -- test_pipeline.py            # Suite de pruebas unitarias para deduplicación
6   -- README.md                  # Instrucciones de despliegue y ejecución
```

Listing 2: Estructura de archivos del proyecto

6. Conclusión

La solución desarrollada satisface la totalidad de los requerimientos técnicos exigidos en la consigna del proyecto: procesa un flujo continuo de datos mediante un productor sintético, aplica conceptos de tiempo de evento y ventanas temporales, elimina duplicados eficientemente y realiza una persistencia de salida idempotente en Apache Kafka.